

第4章 树

建议学时:4-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 掌握树的核心术语:根、父结点、子结点、兄弟、叶、深度、高度、路径
- 完成从 C 的 `first_child + next_sibling` 到 C++ `vector` 子结点容器的迁移
- 掌握前序/中序/后序/层序遍历:递归版、队列版、显式栈版都能写出
- 能用栈从后缀表达式构建表达式树并用中序恢复中缀,与第3章闭环
- 能完整实现BST的 `contains/findMin/insert/remove` 与 AVL 四种旋转,并用 `unique_ptr` 管理内存

💡 从 C 到 C++:本章主题如何迁移

树是"有分支的链表":链表只能排成一条线,树允许一个结点有多个后继,信息分层组织。家族谱、文件目录、语法树都是树。第3章解决"一串数据",本章解决"有层次关系的数据"。

C 里一般树靠双指针 `firstChild + nextSibling` 串起任意多个子结点,通用但笨拙。C++ 复用第3章的 `vector`:子结点放进容器,遍历就是一句范围 `for`,销毁交给 RAII。

本章终点是查找:二分查找 $O(\log n)$,但有序数组插入要搬移元素。BST 把查找、插入、删除同时做到平均 $O(\log n)$;AVL 再把"平均"升级为"保证"。

📦 前置知识自查

数学前置:树高递归式: $\text{height}(\text{空}) = 0, \text{height}(v) = 1 + \max(\text{height}(v.\text{left}), \text{height}(v.\text{right}))$;对数直觉(每次减半 $\rightarrow \log n$)。

C 语言前置: `struct` 与指针、`malloc / free`、递归、`->` 访问成员。

依赖的前面章节:第1章递归(遍历、高度、销毁)、第3章栈与队列(层序用队列,建表达式树用栈;中缀转后缀是 §4.3 的输入)。

🚀 本章学习路线

1. 阶段1(1 小时):§4.1-§4.2 建立术语,上机核对四种遍历输出
2. 阶段2(1 小时):§4.3 表达式树,与中缀转后缀接成闭环
3. 阶段3(1.5 小时):§4.4 手写 BST,默写删除三情形
4. 阶段4(1 小时):§4.5 画图验证 AVL 旋转,通读 §4.6-§4.7
5. 阶段5(实验 1 小时):完成章末实验,复习删除三情形与旋转判定

本章知识导图

- §4.1 树的概念:术语与一般树的实现(根/父/子/兄弟/叶/深度/高度/路径、first_child+next_sibling → vector、一般树 vs 二叉树)
- §4.2 二叉树与四种遍历(递归三序、层序用队列、显式栈模拟递归、高度/结点数/叶数)
- §4.3 表达式树(后缀构建、中序恢复中缀)
- §4.4 二叉查找树 BST(查找/插入/删除、删除三情形、手写 vs set)
- §4.5 AVL 树(平衡因子、LL/RR/LR/RL 旋转、插入后重平衡)
- §4.6 伸展树与 B 树(摊还与局部性、磁盘 IO 与浅而宽)
- §4.7 工程与实践(unique_ptr 内存、std::set/map 对照、调试)

§4.1 树的概念:术语与一般树的实现

4.1.1 术语:根、父结点、子结点、兄弟、叶、深度、高度、路径

树的递归定义:树是结点的集合——要么是空集,要么由一个**根**结点加上若干棵互不相交的**子树**构成,每棵子树本身也是树。这决定了本章算法的递归写法。

图 4-1 一般树的术语

树的递归定义:树 = 根 + 互不相交的子树。



深度从根往下数(根 0),**高度**从叶往上数(叶 0),树高 = 根的高度;**路径**长度 = 边数。n 个结点的树有 n-1 条边。方向相反,勿混淆。

记号说明(树高递归式,全书沿用)

$\text{height}(\text{空}) = 0, \text{height}(v) = 1 + \max(\text{height}(v.\text{left}), \text{height}(v.\text{right}))$ ——它同时是定义、递推式和代码,§4.2 的求高函数就是逐字翻译。

4.1.2 一般树的实现:从 first_child+next_sibling 到 vector 子结点容器

C 靠 firstChild + nextSibling 表达任意多的子结点;C++ 用 vector 保存——第3章 vector 思想在树上的延续:

C 的写法(first_child + next_sibling)	C++ 的写法(vector 子结点容器)
<pre>struct TreeNode { int val; struct TreeNode *firstChild, *nextSibling; };</pre>	<pre>struct TreeNode { int val; vector<TreeNode*> children; };</pre>
遍历子结点:沿 nextSibling 链 while 循环	遍历子结点:范围 for 一句
访问第 k 个子结点要数 k 次指针	children[k] 直接下标访问
插入子结点自己维护链接顺序	children.push_back(c) 一行
忘记 free 子树 = 泄漏	容器自动析构(结点对象仍需管理,见 §4.7)

示例 4-1 一般树结点:vector 子结点容器

```
struct TreeNode {
    int val;
    vector<TreeNode*> children;          // 子结点容器
    explicit TreeNode(int v) : val(v) {}
};

// 对照 C 的 nextSibling 链,范围 for 一句遍历
void printChildren(TreeNode* n) {
    for (TreeNode* c : n->children)
        cout << c->val << ' ';
}
```

4.1.3 一般树 vs 二叉树

二叉树是"每个结点至多两个子结点,且区分左右"的树——只有一个子结点也要说清在左在右。BST、表达式树、堆(第 6 章)都依赖"左/右"语义。一般树与二叉树可互相转换:"长子变左子、兄弟变右子"正是 C 的 first_child+next_sibling。

§4.2 二叉树与四种遍历

4.2.1 二叉树的定义与递归三序遍历

二叉树要么为空,要么由根 + 左子树 + 右子树构成——直接翻译成结点类:

示例 4-2 二叉树结点与前/中/后序递归遍历

```

struct BTreeNode {
    int val;
    BTreeNode* left;
    BTreeNode* right;
    BTreeNode(int v, BTreeNode* l = nullptr, BTreeNode* r = nullptr)
        : val(v), left(l), right(r) {}
};

void preorder(BTreeNode* n) {           // 前序:根 → 左 → 右
    if (n == nullptr) return;
    cout << n->val << ' ';
    preorder(n->left);
    preorder(n->right);
}

void inorder(BTreeNode* n) {           // 中序:左 → 根 → 右
    if (n == nullptr) return;
    inorder(n->left);
    cout << n->val << ' ';
    inorder(n->right);
}

void postorder(BTreeNode* n) {         // 后序:左 → 右 → 根
    if (n == nullptr) return;
    postorder(n->left);
    postorder(n->right);
    cout << n->val << ' ';
}

```

三个函数唯一区别是 `cout` 在两次递归调用的前、中、后——遍历的全部秘密。

例题 4-1 手走一棵二叉树的三种遍历

一棵 BST 形状的二叉树(内部结点 7/3/9,叶 1/6/11):



前序:7 3 1 5 6 9 11;中序:1 3 5 6 7 9 11;后序:1 6 5 3 11 9 7。自查:中序恰好升序,因为它是 BST(\$4.4)。

误区 1:把"访问"与"递归调用"的顺序搞混

打印语句在两次递归之间的位置决定遍历类型——三序唯一的区别就是这行 `cout` 放哪。先用最小树手推一遍再对照。

4.2.2 层序遍历(用队列,呼应第3章)

层序遍历按"从上到下、每层从左到右"访问。递归天然深入、做不到逐层横推,必须用第3章的队列:根先入队,出队访问后左右子结点再入队。

示例 4-3 层序遍历:std::queue

```

#include <queue>
void levelOrder(BTNode* root) {           // 层序:先进先出
    queue<BTNode*> q;
    q.push(root);
    while (!q.empty()) {
        BTNode* n = q.front();
        q.pop();
        if (n == nullptr) continue;
        cout << n->val << ' ';
        q.push(n->left);
        q.push(n->right);
    }
}

```

4.2.3 显式栈模拟递归(C++ 完整版)

任何递归都能改写成"显式栈 + 循环"(第3章)。系统栈压"返回地址 + 局部变量",显式栈压"待处理的结点",不受栈深度限制:

示例 4-4 迭代中序遍历:显式栈

```

void inorderIter(BTNode* root) {          // 显式栈模拟递归中序
    stack<BTNode*> st;
    BTNode* cur = root;
    while (cur != nullptr || !st.empty()) {
        while (cur != nullptr) {         // 1. 沿左链一路压栈
            st.push(cur);
            cur = cur->left;
        }
        cur = st.top();                   // 2. 出栈
        st.pop();
        cout << cur->val << ' ';          // 3. 访问发生在出栈时
        cur = cur->right;                  // 4. 转向右子树
    }
}

```

递归中序也是"先到最左、打印、再进右子树"——迭代版只是自己压栈。

4.2.4 遍历应用:求高度、结点数、叶数

三个经典问题都是"后序思想":先递归算完左右子树,再组合出答案。基准情形都是 `n == nullptr` 返回 0,空树基准让"单个叶结点"自然成立;写树递归先问:空结点返回什么?

示例 4-5 树高、结点数、叶数

```

int height(BTNode* n) { // 空树高度为 0
    if (n == nullptr) return 0;
    return 1 + max(height(n->left), height(n->right));
}
int countNodes(BTNode* n) { // 结点数
    if (n == nullptr) return 0;
    return 1 + countNodes(n->left) + countNodes(n->right);
}
int countLeaves(BTNode* n) { // 叶:左右都为空
    if (n == nullptr) return 0;
    if (n->left == nullptr && n->right == nullptr) return 1;
    return countLeaves(n->left) + countLeaves(n->right);
}

```

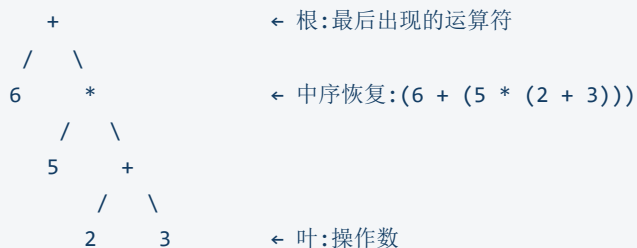
§4.3 表达式树

4.3.1 表达式树的结构

表达式树是编译器处理算术表达式的内部表示: **叶结点是操作数,内部结点是运算符**,左右子树就是两个操作数——后缀式 `6 5 2 3 + * +` 对应:

图 4-2 后缀式 `6 5 2 3 + * +` 的表达式树

表达式树:叶是操作数,内部是运算符。



4.3.2 由后缀表达式构建(用栈,衔接第3章)

构建与第3章后缀求值几乎同构:遇数字建叶结点入栈;遇运算符弹出两棵子树——**先弹的是右操作数**——合并压回。区别只在栈里存数值还是树结点:

示例 4-6 由后缀表达式构建表达式树

```

struct ExprNode {                                // 表达式树结点
    string data;                                // 数字或运算符
    ExprNode* left;
    ExprNode* right;
    ExprNode(const string& d, ExprNode* l = nullptr, ExprNode* r = nullptr)
        : data(d), left(l), right(r) {}
};

ExprNode* buildExprTree(const vector<string>& post) {
    stack<ExprNode*> st;
    for (const string& t : post) {
        if (isdigit(t[0]))
            st.push(new ExprNode(t));    // 数字:建叶结点入栈
        else {
            ExprNode* r = st.top(); st.pop();    // 先弹的是右操作数
            ExprNode* l = st.top(); st.pop();
            st.push(new ExprNode(t, l, r));    // 合并成父结点压回
        }
    }
    return st.top();    // 栈顶即根
}

```

⚠ 误区 2:左右子树接反

弹出时**先弹的是右操作数**。把 `r` 接成左子树, `6 5 -` 就会算成 `5-6`。写完用 `printInfix` 核对带减号的结果。

4.3.3 中序遍历恢复中缀表达式

三序遍历与三种表达式记法一一对应:前序 = 前缀式,中序 = 中缀式,后序 = 后缀式。中序遍历表达式树输出的正是人读的中缀;为优先级自洽,给每个运算符子树加括号:

示例 4-7 中序恢复中缀(带括号)

```

void printInfix(ExprNode* n) {                // 中序恢复中缀
    if (n == nullptr) return;
    bool isOp = !isdigit(n->data[0]);
    if (isOp) cout << '(';                  // 运算符子树加括号
    printInfix(n->left);
    cout << n->data;
    printInfix(n->right);
    if (isOp) cout << ')';
}

```

★ 重点:三序遍历 × 三种表达式 = 一个闭环

第3章"中缀转后缀"把 `(1+2)*3` 变 `12+3*`;本章"后缀建树 + 中序恢复"把它变回 `((1+2)*3)`。三种记法、三种遍历是同一信息的三副面孔。

§4.4 二叉查找树(BST)

4.4.1 BST 性质与 contains/findMin/findMax

二叉查找树(BST)性质: 每个结点的左子树所有值都小于它,右子树所有值都大于它。推论:中序遍历输出严格升序。查找 = 比较后走左或右,一路下行丢掉一半子树——二分查找的树形版,插入删除不用搬移元素:

示例 4-8 BST:contains/findMin/findMax

```
class BST {
    struct Node {
        int val;
        Node* left;
        Node* right;
        Node(int v, Node* l = nullptr, Node* r = nullptr)
            : val(v), left(l), right(r) {}
    };
    Node* root_ = nullptr;
public:
    bool contains(int x) const {          // 查找:沿一条路径下行
        Node* n = root_;
        while (n) {
            if (x == n->val) return true;
            n = (x < n->val) ? n->left : n->right;
        }
        return false;
    }
    int findMin() const {                 // 最小:一路向左
        Node* n = root_;
        while (n->left) n = n->left;
        return n->val;
    }
    int findMax() const {                 // 最大:一路向右
        Node* n = root_;
        while (n->right) n = n->right;
        return n->val;
    }
};
```

4.4.2 插入 insert

C 的做法是"遍历到空位,改父结点指针";C++ 更稳的是**递归返回"新根"**:指针修改收拢在 `return` 里,父结点用返回值接住:

示例 4-9 BST 插入:递归返回新根

```
void insert(int x) { root_ = insert(root_, x); }
private:
static Node* insert(Node* n, int x) {
    if (n == nullptr) return new Node(x);    // 空位:建新结点
    if (x < n->val)    n->left  = insert(n->left, x);
    else if (x > n->val) n->right = insert(n->right, x);
    return n;                                // 相等:忽略(不允许重复)
}
```

4.4.3 删除 remove:三种情形

删除三情形:① **叶**:直接删;② **单子**:让唯一子结点顶替;③ **双子**:用右子树的最小结点顶替被删结点,再删掉它——它大于左子树全部、小于右子树其余,且无左子树:

示例 4-10 BST 删除:三种情形

```
static Node* findMinNode(Node* n) { // 子树中的最小结点
    while (n->left != nullptr) n = n->left;
    return n;
}
static Node* remove(Node* n, int x) {
    if (n == nullptr) return nullptr;
    if (x < n->val) n->left = remove(n->left, x);
    else if (x > n->val) n->right = remove(n->right, x);
    else { // 找到 x
        if (n->left == nullptr) return n->right; // 情形1+2a
        if (n->right == nullptr) return n->left; // 情形2b
        Node* m = findMinNode(n->right); // 情形3:双子
        n->val = m->val; // 复制数据顶替
        n->right = remove(n->right, m->val); // 删掉最小结点
    }
    return n;
}
```

⚠ 误区 3:双子删除的两处经典翻车

① **漏接右子树**:只写 `n->val = m->val` 就 `return n, m` 还挂在右子树里——重复且泄漏,必须再递归删除 `m`。② **删错目标**:写成 `n->right = remove(n->right, x)` 必扑空。先复制,再用 `m` 删右子树。

⚠ 误区 4:删除后悬垂指针

调用方若写 `Node* p = root_; root_ = remove(root_, x); p->val = ...;`——`p` 可能已被删除,访问即未定义行为。删除/修改容器后,旧指针一律视为失效。

4.4.4 复杂度直觉与"手写 vs 标准库"

BST 每个操作沿一条路径下行,路径长度 = 树高。随机插入时树高约 $1.39 \log_2 n$,操作平均 $O(\log n)$;数据**恰好有序**插入时树退化成链,查找变线性扫描。

⚠ 误区 5:把"平均"当"保证"(退化链)

给 BST 喂升序序列,它长成一条只有右孩子的链,查 1 要 $O(n)$ 。这不是理论病:自增主键、日志时间戳等"按序到达"的数据都会触发。解法:① 打乱插入顺序;② 用自平衡结构——AVL (§4.5)、红黑树 (`std::set` 底层,第12章)——把树高**保证**在 $O(\log n)$ 。

工程上需要"有序集合"时直接用标准库,手写 BST 只为理解原理:

示例 4-11 手写 BST 查找 vs `std::set`(并排)

```
// 手写 BST:平均  $O(\log n)$ ,最坏  $O(n)$ 
BST myTree;
myTree.insert(15); myTree.insert(6); myTree.insert(18);
bool a = myTree.contains(6);           // true

// 标准库:std::set 是红黑树(自平衡 BST),同样有序
set<int> st = {15, 6, 18};
bool b = st.count(6);                  // true, $O(\log n)$  且有保证
auto it = st.find(6);                  // 找不到时等于 st.end()
```

C 的手写查找树	C++ 的标准库 set/map
手写 Node 结构 + 递归插入/删除,还要管内存	set<int> / map<K,V> 直接使用,自动管理内存
有序插入退化成链,最坏 $O(n)$	红黑树保证 $O(\log n)$,旋转细节全封装
重复键约定要自己定	count() 返回 0/1,语义明确

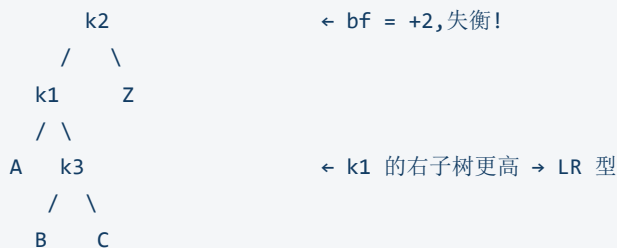
§4.5 AVL 树

4.5.1 平衡因子与失衡类型

AVL 树是第一种自平衡 BST: 每个结点的左右子树高度差至多 1。平衡因子 $bf(v) = height(v.left) - height(v.right)$, 合法值只有 -1、0、+1;插入可能把某祖先推到 ± 2 ,用**旋转**扳回,四种,看失衡结点的哪个孩子更高:

图 4-3 失衡判定:四种类型

$bf(v) = height(v.left) - height(v.right)$,合法取值只有 -1/0/1。



LL:bf=+2 且左重;RR:bf=-2 且右重;LR:bf=+2 且右重;RL:bf=-2 且左重。

记忆法:LL 是"左左",失衡结点和高孩子都在左,一次右旋;LR 是"左右",先下后上。

4.5.2 四种旋转的实现

示例 4-12 AVL 四种旋转

```

int h(Node* n) { return n ? n->height : 0; }

// LL:右旋——左孩子提为根
Node* rotateWithLeftChild(Node* k2) {
    Node* k1 = k2->left;
    k2->left = k1->right;
    k1->right = k2;
    k2->height = 1 + max(h(k2->left), h(k2->right));
    k1->height = 1 + max(h(k1->left), h(k1->right));
    return k1;
}

// RR:左旋,镜像对称
Node* rotateWithRightChild(Node* k1) {
    Node* k2 = k1->right;
    k1->right = k2->left;
    k2->left = k1;
    k1->height = 1 + max(h(k1->left), h(k1->right));
    k2->height = 1 + max(h(k2->left), h(k2->right));
    return k2;
}

// LR:先左旋左孩子,再右旋自己
Node* doubleRotateLeftRight(Node* k3) {
    k3->left = rotateWithRightChild(k3->left);
    return rotateWithLeftChild(k3);
}

// RL:先右旋右孩子,再左旋自己
Node* doubleRotateRightLeft(Node* k1) {
    k1->right = rotateWithLeftChild(k1->right);
    return rotateWithRightChild(k1);
}

```

★ 重点:旋转的口诀

单旋转(LL/RR)是"把高孩子提上来当根,自己让位,多余的一支过继"——只改三个指针, $O(1)$;双旋转(LR/RL)是"先捋直再旋转"。**旋转不破坏 BST 的大小序**——中序遍历结果不变。

4.5.3 插入后的重平衡与复杂度对照

AVL 插入 = BST 插入 + 沿路径回溯:每层更新高度、检查平衡因子。插入只改变一条路径,第一个失衡点旋转一次即恢复平衡:

示例 4-13 AVL 插入:更新高度 + 失衡判定

```

Node* insert(Node* n, int x) {
    if (n == nullptr) return new Node(x);
    if (x < n->val)      n->left  = insert(n->left, x);
    else if (x > n->val) n->right = insert(n->right, x);
    n->height = 1 + max(h(n->left), h(n->right)); // 更新高度
    int bf = h(n->left) - h(n->right);          // 平衡因子
    if (bf > 1) {                               // 左子树高:LL 或 LR
        if (x < n->left->val) return rotateWithLeftChild(n); // LL
        return doubleRotateLeftRight(n);                  // LR
    }
    if (bf < -1) {                               // 右子树高:RR 或 RL
        if (x > n->right->val) return rotateWithRightChild(n); // RR
        return doubleRotateRightLeft(n);                    // RL
    }
    return n; // 本结点仍平衡
}

```

判定 LL/LR:看 x 落在失衡结点左孩子的哪一侧, $x < n->left->val$ 是 LL,否则 LR。

例题 4-2(手走旋转):空树依次插入 3、2、1:结点 3 的 $bf=2$ 且落在左孩子 2 的左子树 $\rightarrow$ LL,右旋:2 提为根,3 变其右孩子;再插 4、5,结点 3 的 $bf=-2 \rightarrow$ RR。最终:根 2,左 1,右 4(左 3 右 5)。

朴素 BST	AVL 树
查找/插入/删除平均 $O(\log n)$	查找/插入/删除 保证 $O(\log n)$
有序输入退化成链,最坏 $O(n)$	任意输入序列,树高 $\leq 1.44 \log_2 n$
实现简单,无额外空间	每个结点多存一个高度(或平衡因子)
插入/删除后不检查平衡	插入/删除后沿路径旋转,常数更大

§4.6 伸展树与 B 树

4.6.1 伸展树:摊还与访问局部性

伸展树(splay tree)的动机是 **访问局部性:刚被访问的数据往往很快再次被访问** (缓存、词频查询、符号表)。AVL 用平衡因子保证每次 $O(\log n)$,代价是代码复杂;伸展树不要平衡因子,只约定:每次访问后把该结点"伸展"到根。

结论(证明在第11章摊还分析):单次最坏 $O(n)$,连续 m 次操作总代价 $O(m \log n)$, **摊还 $O(\log n)$** ;反复访问的结点总停在根附近,但单次最坏延迟无法保证。

4.6.2 B 树:磁盘 IO 与"浅而宽"

B 树的动机是磁盘:磁盘随机读约 10 ms,内存访问是纳秒级,差约 10 万倍,数据库索引里 **决定性能的不是比较次数,而是下潜层数(= IO 次数)**。BST 树高约 $\log_2 n$, 10^7 条记录约 23 层;B 树是"M 路分支",树高降到 $\log_M n$,取 $M=1000$ 只需 3 层——**"浅而宽"把 IO 从 20 多压到 3**。

结论:B 树查找/插入/删除都是 $O(\log_M n)$;分裂/合并细节不展开,工程上按"一个结点装满一个磁盘块"定 M 。SQLite 的 B+ 树等索引都是它的工业后代。

§4.7 工程与实践

4.7.1 用 unique_ptr 建树,告别手动 delete

裸指针树有**泄漏**、**双删**、**深树爆栈**三个问题。 `unique_ptr` 一次解决:结点"拥有"子树,结点销毁时子树自动递归销毁:

示例 4-14 `unique_ptr` 树:析构自动递归

```
#include <memory>
struct BSTNode {
    int val;
    unique_ptr<BSTNode> left, right;    // 结点拥有子树
    explicit BSTNode(int v) : val(v) {}
};

unique_ptr<BSTNode> insert(unique_ptr<BSTNode> n, int x) {
    if (!n) return make_unique<BSTNode>(x);    // 空位:构造新结点
    if (x < n->val)    n->left = insert(move(n->left), x);
    else if (x > n->val) n->right = insert(move(n->right), x);
    return n;
}
// 根离开作用域:unique_ptr 依次销毁左右子树,递归自动完成
```

⚠ 误区 6:裸指针递归析构的坑

手写 `~BST() { delete root_; }` 时析构是递归的:退化链递归 n 层直接撞栈上限;共享结点会 `double free`。用 `unique_ptr` 后:① 深树销毁仍可能爆栈,可改显式栈迭代销毁(§4.2.3);② 禁止拷贝,共享树换 `shared_ptr`。先想清楚"谁拥有这个结点",再决定用什么指针。

C 的 malloc/free 树	C++ 的 unique_ptr 树
<code>node = malloc(...)</code> ,用完 <code>free</code> ,漏一次就泄漏	<code>make_unique<Node>(...)</code> ,离开作用域自动释放
销毁整树要手写递归 <code>delete</code> ,深树爆栈、共享双删	析构自动递归;禁止拷贝从类型上排除双删
释放后指针仍存在,悬垂未定义行为	释放后 <code>unique_ptr</code> 自动置空, <code>!p</code> 判空安全

4.7.2 标准库对照:std::set/map 是红黑树

工程上"有序集合/有序映射"直接用标准库: `std::set`、`std::map` 底层都是**红黑树**——比 AVL 更宽松的自平衡 BST(旋转更少),操作保证 $O(\log n)$,迭代器按序输出。第12章细讲,先建立" `set/map` = 自平衡 BST,要字典序或范围查询就选它"的直觉。

示例 4-15 `std::map` 做词频统计

```

#include <map>
map<string, int> freq;           // 红黑树:key 有序,  $O(\log n)$ 
for (const string& w : words)
    ++freq[w];                  // 不存在时自动初始化为 0
for (const auto& p : freq)      // 遍历 = 中序 = 字典序
    cout << p.first << ": " << p.second << '\n';
int cnt = freq["the"];          // 查找  $O(\log n)$ 
freq.erase("dog");              // 删除  $O(\log n)$ 

```

4.7.3 调试技巧:打印树与验证性质

- **打印整棵树**:写带缩进的递归打印,比逐结点断点高效
- **中序必升序**:BST 的中序遍历必须严格升序——最便宜的全局检查
- **验证 AVL**:插入后断言全部结点 $|bf| \leq 1$ 并打印树高

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
查找	BST	$O(\log n)$	$O(n)$
插入	BST	$O(\log n)$	$O(n)$
删除	BST	$O(\log n)$	$O(n)$
查找最小 / 最大	BST	$O(\log n)$	$O(n)$
查找 / 插入 / 删除	AVL 树	$O(\log n)$	$O(\log n)$
前/中/后序遍历	二叉树	$O(n)$	$O(n)$
层序遍历	二叉树	$O(n)$	$O(n)$
求高度 / 结点数 / 叶数	二叉树(后序递归)	$O(n)$	$O(n)$
构建表达式树	后缀式 + 栈	$O(n)$	$O(n)$
恢复中缀表达式	表达式树中序	$O(n)$	$O(n)$
查找 / 插入 / 删除	伸展树	$O(\log n)$	$O(n)$ (单次)
查找 / 插入 / 删除	B 树	$O(\log_M n)$	$O(\log_M n)$
查找	有序数组(二分)	$O(\log n)$	$O(\log n)$
插入(有序数组中)	动态数组	$O(n)$	$O(n)$
按值查找	无序数组/链表	$O(n)$	$O(n)$
查找 / 插入 / 删除	set/map 红黑树	$O(\log n)$	$O(\log n)$

最小必会清单

- 能说出树的核心术语,解释深度与高度的数法
- 能默写 vector 子结点表示,对照 first_child+next_sibling
- 能写出前/中/后序遍历递归版,说清打印位置的作用

- 能写出层序遍历(队列版),解释为什么递归做不到层序
- 能写出求高度/结点数/叶数的递归函数,说明空树基准
- 能用栈从后缀构建表达式树(先弹右操作数),再中序恢复中缀
- 能默写 BST 的 contains/findMin/insert/remove,说清删除三情形
- 能用 unique_ptr 建树;能说出 std::set/map 底层是红黑树

自测题

Q 1. 一棵有 n 个结点的一般树有几条边?树的深度与高度有什么区别?

✓ 参考答案

$n-1$ 条:除根外每个结点都有一个父结点。深度从根往下(根 0),高度从叶往上(叶 0);树高 = 根的高度。

Q 2. 对例题 4-1 的树,写出四种遍历输出。

✓ 参考答案

前序:7 3 1 5 6 9 11;中序:1 3 5 6 7 9 11;后序:1 6 5 3 11 9 7;层序:7 3 9 1 5 11 6。后序 11 在 9 前:右子树先于根。

Q 3. 后缀式 $6\ 5\ 2\ -\ 3\ *\ +$ 的表达式树,中序恢复(带括号)是什么?

✓ 参考答案

构建:6、5、2 入栈;- 弹 2、5 得 $(5-2)$;3 入栈;* 得 $(5-2)*3$;+ 得 $6+((5-2)*3)$ 。先弹的是右操作数,故是 $5-2$ 而非 $2-5$ 。

Q 4. BST 删除双子结点时,为什么取右子树最小结点顶替?取左子树最大结点可以吗?

✓ 参考答案

右子树最小结点 m 大于左子树全部、小于右子树其余,顶替后性质不变;且 m 没有左子树,删除退化为单子/叶情形。取左子树最大结点也可行。

Q 5. 空树依次插入 5、3、7、2、4、6、8、1,哪一步触发旋转?什么类型?旋转后树根是谁?

✓ 参考答案

插入 1 后结点 5 的 $bf=+2$,1 在 5 的左孩子 3 的左子树 $\rightarrow LL$,右旋:3 提为根,5 变其右孩子。旋转后:根 3,左 2(左 1 右 4),右 7(左 6 右 8)。

章末实验题:单词频次统计器(BST 实现)

实验 4:单词频次统计器(BST 实现)

实验目标:用自写 BST 实现命令行单词频次统计器:统计词频、查找、删除,并报告树高与平衡性。

实验要求:

- 任务 1:读入多行英文文本,把连续字母序列切分为单词并统一转小写(\$4.1 树结构、字符串遍历)
- 任务 2:用自写 BST 统计词频——结点存"单词 + 频次",已存在的单词频次 +1(\$4.4 insert)
- 任务 3:中序遍历按字典序输出词频表(\$4.2 中序遍历)
- 任务 4:查找指定单词频次;删除某单词后重新遍历,验证查找结果为 0(\$4.4 contains/remove 三情形)
- 任务 5(加分):平衡检查并报告树高(\$4.5 平衡因子、\$4.2 求高度)

实验环境:C++17。编译运行: `g++ -std=c++17 lab4.cpp -o lab4 && ./lab4`。

第一步 实现结点结构与 insert/find,验证频次正确后再补中序遍历——增量开发

第二步 除用示例 4-10 的三情形骨架;双子先复制右子树最小结点,再删除它(见误区 3)

第三步 点内存用 `unique_ptr` (示例 4-14),类里就不需要析构函数

✔ 参考答案(完整可运行程序,分三段)

```

// lab4.cpp — 单词频次统计器(BST 实现)(第一段:结点与核心操作)
#include <iostream>
#include <string>
#include <memory>
#include <cctype>
#include <algorithm>
using namespace std;

struct BSTNode {                                // 单词 + 频次 + 左右子树
    string word;
    int count;
    unique_ptr<BSTNode> left, right;
    explicit BSTNode(const string& w) : word(w), count(1) {}
};

class WordBST {
public:
    void insert(const string& w) { root_ = insert(move(root_), w); }
    int find(const string& w) const {                // 查找频次,不存在返回 0
        BSTNode* n = root_.get();
        while (n) {
            if (w == n->word) return n->count;
            n = (w < n->word) ? n->left.get() : n->right.get();
        }
        return 0;
    }
    void remove(const string& w) { root_ = remove(move(root_), w); }
    BSTNode* root() const { return root_.get(); }    // 供第二段辅助函数
private:
    unique_ptr<BSTNode> root_;
    static unique_ptr<BSTNode> insert(unique_ptr<BSTNode> n, const string& w) {
        if (!n) return make_unique<BSTNode>(w);    // 空位:新建结点
        if (w < n->word) n->left = insert(move(n->left), w);
        else if (w > n->word) n->right = insert(move(n->right), w);
        else ++n->count;                            // 已存在:频次 +1
        return n;
    }
    static BSTNode* findMin(BSTNode* n) {           // 右子树最小结点
        while (n->left) n = n->left.get();
        return n;
    }
    static unique_ptr<BSTNode> removeMin(unique_ptr<BSTNode> n) {
        if (!n->left) return move(n->right);
        n->left = removeMin(move(n->left));
        return n;
    }
    static unique_ptr<BSTNode> remove(unique_ptr<BSTNode> n, const string& w) {
        if (!n) return nullptr;                    // 单词不存在,忽略
        if (w < n->word) n->left = remove(move(n->left), w);
        else if (w > n->word) n->right = remove(move(n->right), w);
        else {                                       // 找到目标结点
            if (!n->left) return move(n->right);    // 叶或只有右子树
            if (!n->right) return move(n->left);    // 只有左子树
            BSTNode* m = findMin(n->right.get()); // 双子:取右子树最小
            n->word = m->word;                        // 复制数据顶替
            n->count = m->count;
        }
    }
};

```

```

        n->right = removeMin(move(n->right));
    }
    return n;
}
};

```

lab4.cpp(第二段:中序、树高与平衡辅助函数)

```

// (第二段:中序遍历、树高与平衡检查—辅助函数,配合 tree.root())
#include <utility>
void inorder(BSTNode* n) {           // 中序 = 字典序
    if (!n) return;
    inorder(n->left.get());
    cout << n->word << '\t' << n->count << '\n';
    inorder(n->right.get());
}
int height(BSTNode* n) {             // 空树高度为 0
    if (!n) return 0;
    return 1 + max(height(n->left.get()), height(n->right.get()));
}
pair<int, bool> check(BSTNode* n) {  // 返回(高度, 是否平衡)
    if (!n) return {0, true};
    auto [hl, bl] = check(n->left.get());
    auto [hr, br] = check(n->right.get());
    return {1 + max(hl, hr), bl && br && abs(hl - hr) <= 1};
}

```

lab4.cpp(第三段:主程序)

```

int main() {
    cout << "单词频次统计器(BST 实现)—输入英文文本,空行结束:\n";
    string line, text;
    while (getline(cin, line)) {
        if (line.empty()) break;
        text += line + '\n';
    }

    WordBST tree;
    string word;
    for (char c : text) {
        if (isalpha((unsigned char)c))
            word += (char)tolower((unsigned char)c);
        else if (!word.empty()) {
            tree.insert(word);
            word.clear();
        }
    }
    if (!word.empty()) tree.insert(word);

    cout << "\n--- 任务 3:中序遍历输出词频表(字典序)---\n";
    inorder(tree.root());

    cout << "\n--- 任务 4:查找频次 ---\n";
    for (const string& w : {"the", "dog", "hello"})
        cout << "查找 '" << w << "': 频次 = " << tree.find(w) << '\n';

    cout << "\n删除 'dog' 后重新遍历:\n";
    tree.remove("dog");
    cout << "删除后 'dog' 频次 = " << tree.find("dog") << "(应为 0)\n";
    inorder(tree.root());

    cout << "\n--- 任务 5(加分):树高与平衡检查 ---\n";
    cout << "树高 = " << height(tree.root())
        << ", 是否平衡 = " << (check(tree.root()).second ? "是" : "否") << '\n';
    return 0;
}

```

示例输入输出(输入文本:The quick brown fox jumps over the lazy dog and the dog barks at the cat while the cat sleeps on the warm mat):

单词频次统计器(BST 实现)—输入英文文本,空行结束:

--- 任务 3:中序遍历输出词频表(字典序)---

```
and      1
at       1
barks    1
brown    1
cat      2
dog      2
fox      1
jumps    1
lazy     1
mat      1
on       1
over     1
quick    1
sleeps   1
the      6
warm     1
while    1
```

--- 任务 4:查找频次 ---

查找 'the': 频次 = 6

查找 'dog': 频次 = 2

查找 'hello': 频次 = 0

删除 'dog' 后重新遍历:

删除后 'dog' 频次 = 0(应为 0)

```
and      1
at       1
barks    1
brown    1
cat      2
fox      1
jumps    1
lazy     1
mat      1
on       1
over     1
quick    1
sleeps   1
the      6
warm     1
while    1
```

--- 任务 5(加分):树高与平衡检查 ---

树高 = 9, 是否平衡 = 否

设计说明:① 任务覆盖本章主线:分词(\$4.1)、插入/删除三情形(\$4.4)、中序遍历(\$4.2)、平衡检查(\$4.5),串成可运行流水线;② 结点用 `unique_ptr` 持有子树,类无需析构函数,双子删除 `return move(n->left)` 即释放孤儿结点(\$4.7);③ 平衡检查是"返回(高度,是否平衡)"的自底向上函数,一次递归算两件事;实测树高 9,AVL 可压到约 6。

下一章预告:第5章 散列——把查找从 $O(\log n)$ 进一步加速到平均 $O(1)$;散列表的分离链接法将使用第3章的链表作为桶,而"何时用BST、何时用散列"的选型判断将贯穿第5章与后续工程实践。